

NETFLIX · STREAMING

Staying up when the cloud went down

How Netflix designed for failure, and kept streaming through a major AWS outage that took many other sites down, plus how to get the same resilience without a chaos-engineering team.

Apr 2011

major AWS US-East outage

Stayed up

no major customer-facing outage

2012

Chaos Monkey open-sourced

Industry	Consumer streaming
Problem	Running a business-critical service on cloud infrastructure that can and does fail
Approach	Assume failure: redundancy across zones, graceful degradation, and failure injected on purpose
Source	Netflix Technology Blog, April 2011 [1]

01 The challenge

In April 2011 a major outage in Amazon Web Services' US-East region disrupted many well-known sites. Any company running there had to assume that parts of its infrastructure could disappear without warning, and that the question was not whether but when.

02 What Netflix did

- **Designed for failure.** Services were spread across several availability zones, so losing one did not take the product down.
- **Degraded gracefully.** When a dependency failed, the service fell back to a simpler experience instead of failing outright.
- **Broke things on purpose.** Chaos Monkey randomly switched off production servers during working hours, so engineers built services that recovered automatically and practised recovery routinely.
- **Learned from the outage.** Netflix's own write-up named the gaps the outage exposed, including handling the loss of a whole zone, and fed them back into tooling [1][2].

03 The results

Netflix reported that it kept streaming through the April 2011 outage without a major customer-facing outage, while it acknowledged where its systems needed to improve [1]. Chaos Monkey was later open-sourced (2012), and the practice it started became known as chaos engineering.

The lesson: reliability comes from rehearsal. Systems that are regularly made to fail, on your schedule, recover when failure arrives on its own.

04 At your scale: a 90-day plan

You don't need a chaos-engineering team. A handful of deliberate practices give a mid-size product most of the resilience.

Days 1–30 · Define "up"	Days 31–60 · Remove single points	Days 61–90 · Rehearse
• Pick the customer journey that pays the bills (sign-up, checkout, login) • Set an SLO for it, e.g. 99.9% successful requests • Alert on SLO burn, not on every CPU spike	• Run across two or more availability zones behind a load balancer • Health checks and autoscaling on every service • Managed databases with automated failover and tested backups	• First game day: switch off one component on purpose, in staging • Write runbooks for what you learned • Repeat quarterly, then carefully in production

05 Readiness checklist

- | | |
|--|--|
| ☐ You have a written SLO for your key journey | ☐ Alerts fire on customer impact, not noise |
| ☐ Losing one server doesn't page anyone | ☐ There's an on-call rota and runbooks |
| ☐ Losing one zone doesn't take you down | ☐ Incidents get a blameless review |
| ☐ Backups have been restored, not just taken | ☐ You've switched something off on purpose this quarter |

Want this plan for your team?

A 30-minute call: we'll find your single points of failure and tailor the 90 days.

[Book a discovery call](#)

Sources

1. Netflix Technology Blog, "Lessons Netflix Learned from the AWS Outage" (April 2011): netflixtechblog.com
2. Netflix Technology Blog, "The Netflix Simian Army" (July 2011): netflixtechblog.com

Based on publicly available information. Netflix is not an eXcommerce client, and this document is not endorsed by Netflix. The 90-day plan and checklist are eXcommerce's recommendations. © eXcommerce LLP · excommerce.online